



University of
Sheffield



A World
TOP 100
University



**UNIVERSITY
OF THE YEAR**

Mutation Testing

Software Testing (COM3529)

Checkin code: BC-CV-VO

José Rojas & Neil Walkinshaw

Week 8

Roadmap

Beizer's Maturity Model

Test Automation

Unit Testing

Control / Data-Flow Analysis

Code Coverage

Regression Testing

Model-Based Testing

Mutation Testing

Search-Based Test Generation

Fuzzing

Roadmap

Beizer's Maturity Model

Test Automation

Unit Testing

Control / Data-Flow Analysis

Code Coverage

Regression Testing

Model-Based Testing

Mutation Testing

Search-Based Test Generation

Fuzzing

A hand holding a pen over a notebook with a blue overlay.

Motivation

Motivation

Coverage criteria are fallible.

Even if you achieve 100% code coverage, your tests might miss crucial faults.

A good pre-condition for a good test set...

... but a poor measure of how good it is.

Coverage Is Not Strongly Correlated with Test Suite Effectiveness

Laura Inozemtseva and Reid Holmes
School of Computer Science
University of Waterloo
Waterloo, ON, Canada
{linozem,rtholmes}@uwaterloo.ca

ABSTRACT

The coverage of a test suite is often used as a proxy for its ability to detect faults. However, previous studies that investigated the correlation between code coverage and test suite effectiveness have failed to reach a consensus about the nature and strength of the relationship between these test suite characteristics. Moreover, many of the studies were done with small or synthetic programs, making it unclear whether their results generalize to larger programs, and some of the studies did not account for the confounding influence of test suite size. In addition, most of the studies were done with adequate suites, which are rare in practice, so the results may not generalize to typical test suites.

We have extended these studies by evaluating the relation-

1. INTRODUCTION

Testing is an important part of producing high quality software, but its effectiveness depends on the quality of the test suite: some suites are better at detecting faults than others. Naturally, developers want their test suites to be good at exposing faults, necessitating a method for measuring the fault detection effectiveness of a test suite. Testing textbooks often recommend coverage as one of the metrics that can be used for this purpose (e.g., [29,34]). This is intuitively appealing, since it is clear that a test suite cannot find bugs in code it never executes; it is also supported by studies that have found a relationship between code coverage and fault detection effectiveness [3,6,14-17,24,31,39].

Unfortunately, these studies do not agree on the strength

Motivation

Coverage criteria are fallible.

Even if you achieve 100% code coverage, your tests might miss crucial faults.

A good pre-condition for a good test set...

... but a poor measure of how good it is.

Lots of research that demonstrates this.

Coverage Is Not Strongly Correlated with Test Suite Effectiveness

Laura Inozemtseva and Reid Holmes
School of Computer Science
University of Waterloo
Waterloo, ON, Canada
{linozem,rtholmes}@uwaterloo.ca

ABSTRACT

The coverage of a test suite is often used as a proxy for its ability to detect faults. However, previous studies that investigated the correlation between code coverage and test suite effectiveness have failed to reach a consensus about the nature and strength of the relationship between these test suite characteristics. Moreover, many of the studies were done with small or synthetic programs, making it unclear whether their results generalize to larger programs, and some of the studies did not account for the confounding influence of test suite size. In addition, most of the studies were done with adequate suites, which are rare in practice, so the results may not generalize to typical test suites.

We have extended these studies by evaluating the relation-

1. INTRODUCTION

Testing is an important part of producing high quality software, but its effectiveness depends on the quality of the test suite: some suites are better at detecting faults than others. Naturally, developers want their test suites to be good at exposing faults, necessitating a method for measuring the fault detection effectiveness of a test suite. Testing textbooks often recommend coverage as one of the metrics that can be used for this purpose (e.g., [29,34]). This is intuitively appealing, since it is clear that a test suite cannot find bugs in code it never executes; it is also supported by studies that have found a relationship between code coverage and fault detection effectiveness [3,6,14-17,24,31,39].

Unfortunately, these studies do not agree on the strength

Motivation

Coverage criteria are fallible.

Even if you achieve 100% code coverage, your tests might miss crucial faults.

A good pre-condition for a good test set...

... but a poor measure of how good it is.

Lots of research that demonstrates this.

How can we demonstrate that a test set really is adequate?

Coverage Is Not Strongly Correlated with Test Suite Effectiveness

Laura Inozemtseva and Reid Holmes
School of Computer Science
University of Waterloo
Waterloo, ON, Canada
{lminozem,rtholmes}@uwaterloo.ca

ABSTRACT

The coverage of a test suite is often used as a proxy for its ability to detect faults. However, previous studies that investigated the correlation between code coverage and test suite effectiveness have failed to reach a consensus about the nature and strength of the relationship between these test suite characteristics. Moreover, many of the studies were done with small or synthetic programs, making it unclear whether their results generalize to larger programs, and some of the studies did not account for the confounding influence of test suite size. In addition, most of the studies were done with adequate suites, which are rare in practice, so the results may not generalize to typical test suites.

We have extended these studies by evaluating the relation-

1. INTRODUCTION

Testing is an important part of producing high quality software, but its effectiveness depends on the quality of the test suite: some suites are better at detecting faults than others. Naturally, developers want their test suites to be good at exposing faults, necessitating a method for measuring the fault detection effectiveness of a test suite. Testing textbooks often recommend coverage as one of the metrics that can be used for this purpose (e.g., [29,34]). This is intuitively appealing, since it is clear that a test suite cannot find bugs in code it never executes; it is also supported by studies that have found a relationship between code coverage and fault detection effectiveness [3,6,14-17,24,31,39].

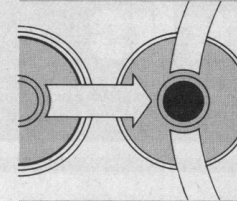
Unfortunately, these studies do not agree on the strength

Competent Programmer Hypothesis

Hints on Test Data Selection: Help for the Practicing Programmer

Richard A. DeMillo
Georgia Institute of Technology

Richard J. Lipton and Frederick G. Sayward
Yale University



In many cases tests of a program that uncover simple errors are also effective in uncovering much more complex errors. This so-called coupling effect can be used to save work during the testing process.

Much of the technical literature in software reliability deals with tentative methodologies and underdeveloped techniques; hence it is not surprising that the programming staff responsible for debugging a large piece of software often feels ignored. It is an economic and political requirement in most production programming shops that programmers shall spend as little time as possible in testing. The programmer must therefore be content to test cleverly but cheaply; state-of-the-art methodologies always seem to be just beyond what can be afforded. We intend to convince the reader that much can be accomplished even under these constraints.

From the point of view of management, there is some justification for opposing a long-term view of the testing phase of the development cycle. [Figure 1](#) shows the relative effect of testing on the remaining system bugs for several medium-scale systems

How, then, should programmers cope? Their more sophisticated general methodologies are not likely to be applicable.² In addition, they have the burden of convincing managers that their software is indeed reliable.

The coupling effect

Programmers, however, have one great advantage that is almost never really exploited: they create programs that are *close* to being correct! Programmers do not create programs at random; competent programmers, in their many iterations through the design process, are constantly whittling away the distance between what their programs look like now and what they are intended to look like. Programmers also have at their disposal

Competent Programmer Hypothesis

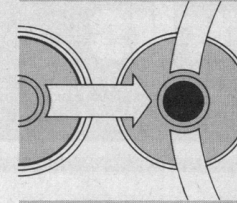
“Programmers will tend to create programs that are *close* to being correct!”

A realistic fault can be characterised as one or more very minor changes to the source code.

Hints on Test Data Selection: Help for the Practicing Programmer

Richard A. DeMillo
Georgia Institute of Technology

Richard J. Lipton and Frederick G. Sayward
Yale University



In many cases tests of a program that uncover simple errors are also effective in uncovering much more complex errors. This so-called coupling effect can be used to save work during the testing process.

Much of the technical literature in software reliability deals with tentative methodologies and underdeveloped techniques; hence it is not surprising that the programming staff responsible for debugging a large piece of software often feels ignored. It is an economic and political requirement in most production programming shops that programmers shall spend as little time as possible in testing. The programmer must therefore be content to test cleverly but cheaply; state-of-the-art methodologies always seem to be just beyond what can be afforded. We intend to convince the reader that much can be accomplished even under these constraints.

From the point of view of management, there is some justification for opposing a long-term view of the testing phase of the development cycle. Figure 1 shows the relative effect of testing on the remaining system bugs for several medium-scale systems

How, then, should programmers cope? Their more sophisticated general methodologies are not likely to be applicable.² In addition, they have the burden of convincing managers that their software is indeed reliable.

The coupling effect

Programmers, however, have one great advantage that is almost never really exploited: they create programs that are *close* to being correct! Programmers do not create programs at random; competent programmers, in their many iterations through the design process, are constantly whittling away the distance between what their programs look like now and what they are intended to look like. Programmers also have at their disposal

Competent Programmer Hypothesis

“Programmers will tend to create programs that are *close* to being correct!”

A realistic fault can be characterised as one or more very minor changes to the source code.

Corroborated by evidence.

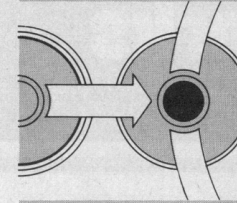
Bug fixes tend to involve targeted changes to specific lines of code.

Less common to incorporate contiguous segments of code.

Hints on Test Data Selection: Help for the Practicing Programmer

Richard A. DeMillo
Georgia Institute of Technology

Richard J. Lipton and Frederick G. Sayward
Yale University



In many cases tests of a program that uncover simple errors are also effective in uncovering much more complex errors. This so-called coupling effect can be used to save work during the testing process.

Much of the technical literature in software reliability deals with tentative methodologies and underdeveloped techniques; hence it is not surprising that the programming staff responsible for debugging a large piece of software often feels ignored. It is an economic and political requirement in most production programming shops that programmers shall spend as little time as possible in testing. The programmer must therefore be content to test cleverly but cheaply; state-of-the-art methodologies always seem to be just beyond what can be afforded. We intend to convince the reader that much can be accomplished even under these constraints.

From the point of view of management, there is some justification for opposing a long-term view of the testing phase of the development cycle. Figure 1 shows the relative effect of testing on the remaining system bugs for several medium-scale systems

How, then, should programmers cope? Their more sophisticated general methodologies are not likely to be applicable.² In addition, they have the burden of convincing managers that their software is indeed reliable.

The coupling effect

Programmers, however, have one great advantage that is almost never really exploited: they create programs that are *close* to being correct! Programmers do not create programs at random; competent programmers, in their many iterations through the design process, are constantly whittling away the distance between what their programs look like now and what they are intended to look like. Programmers also have at their disposal

Competent Programmer Hypothesis

“Programmers will tend to create programs that are *close* to being correct!”

A realistic fault can be characterised as one or more very minor changes to the source code.

Corroborated by evidence.

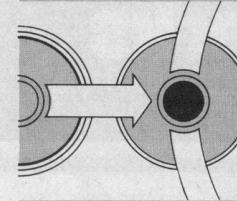
Bug fixes tend to involve targeted changes to specific lines of code.

Less common to incorporate contiguous segments of code.

Hints on Test Data Selection: Help for the Practicing Programmer

Richard A. DeMillo
Georgia Institute of Technology

Richard J. Lipton and Frederick G. Sayward
Yale University



In many cases tests of a program that uncover simple errors are also effective in uncovering much more complex errors. This so-called coupling effect can be used to save work during the testing process.

Much of the technical literature in software reliability deals with tentative methodologies and underdeveloped techniques; hence it is not surprising that the programming staff responsible for debugging a large piece of software often feels ignored. It is an economic and political requirement in most production programming shops that programmers shall spend as little time as possible in testing. The programmer must therefore be content to test cleverly but cheaply; state-of-the-art methodologies always seem to be just beyond what can be afforded. We intend to convince the reader that much can be accomplished even under these constraints.

From the point of view of management, there is some justification for opposing a long-term view of the testing phase of the development cycle. [Figure 1](#) shows the relative effect of testing on the remaining system bugs for several medium-scale systems

How, then, should programmers cope? Their more sophisticated general methodologies are not likely to be applicable.² In addition, they have the burden of convincing managers that their software is indeed reliable.

The coupling effect

Programmers, however, have one great advantage that is almost never really exploited: they create programs that are *close* to being correct! Programmers do not create programs at random; competent programmers, in their many iterations through the design process, are constantly whittling away the distance between what their programs look like now and what they are intended to look like. Programmers also have at their disposal

Competent Programmer Hypothesis

“Programmers will tend to create programs that are *close* to being correct!”

A realistic fault can be characterised as one or more very minor changes to the source code.

Corroborated by evidence.

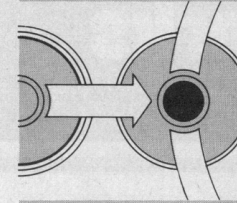
Bug fixes tend to involve targeted changes to specific lines of code.

Less common to incorporate contiguous segments of code.

Hints on Test Data Selection: Help for the Practicing Programmer

Richard A. DeMillo
Georgia Institute of Technology

Richard J. Lipton and Frederick G. Sayward
Yale University



In many cases tests of a program that uncover simple errors are also effective in uncovering much more complex errors. This so-called coupling effect can be used to save work during the testing process.

Much of the technical literature in software reliability deals with tentative methodologies and underdeveloped techniques; hence it is not surprising that the programming staff responsible for debugging a large piece of software often feels ignored. It is an economic and political requirement in most production programming shops that programmers shall spend as little time as possible in testing. The programmer must therefore be content to test cleverly but cheaply; state-of-the-art methodologies always seem to be just beyond what can be afforded. We intend to convince the reader that much can be accomplished even under these constraints.

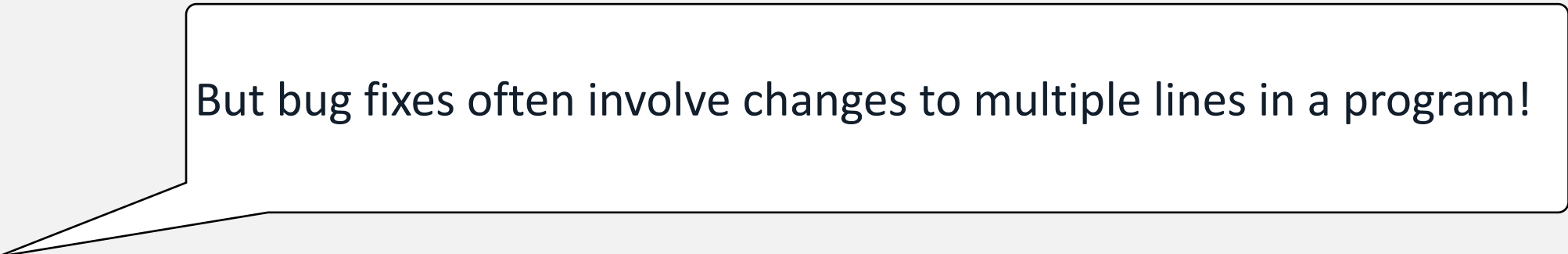
From the point of view of management, there is some justification for opposing a long-term view of the testing phase of the development cycle. [Figure 1](#) shows the relative effect of testing on the remaining system bugs for several medium-scale systems

How, then, should programmers cope? Their more sophisticated general methodologies are not likely to be applicable.² In addition, they have the burden of convincing managers that their software is indeed reliable.

The coupling effect

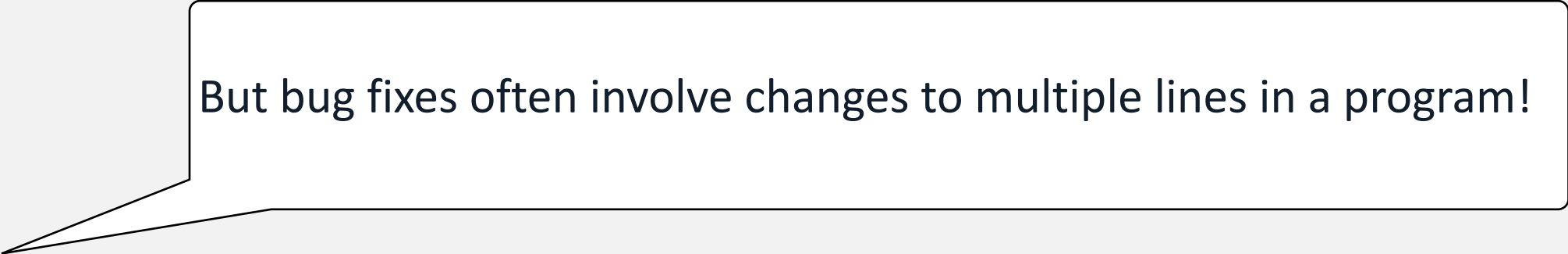
Programmers, however, have one great advantage that is almost never really exploited: they create programs that are *close* to being correct! Programmers do not create programs at random; competent programmers, in their many iterations through the design process, are constantly whittling away the distance between what their programs look like now and what they are intended to look like. Programmers also have at their disposal

The Coupling Effect



But bug fixes often involve changes to multiple lines in a program!

The Coupling Effect



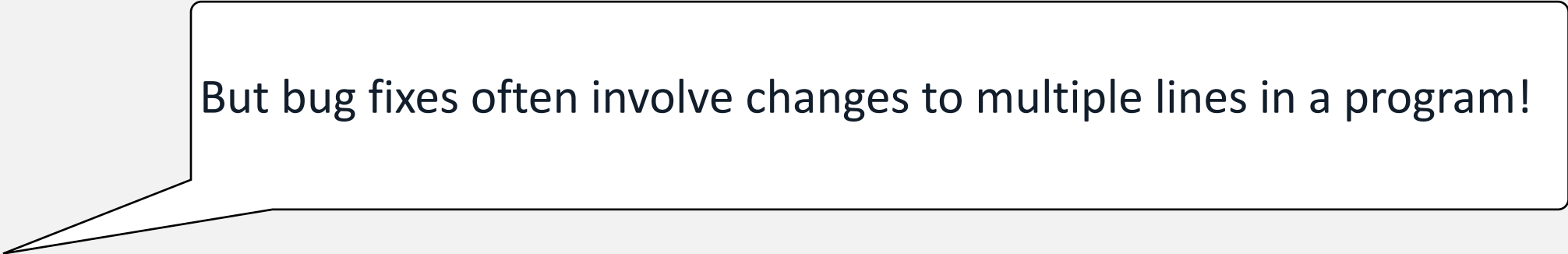
But bug fixes often involve changes to multiple lines in a program!

Correct, but these changes tend to be *spread around* the source code.

Complex faults consist of lots of “simple” localised faults.

They are not contiguous chunks of code.

The Coupling Effect



But bug fixes often involve changes to multiple lines in a program!

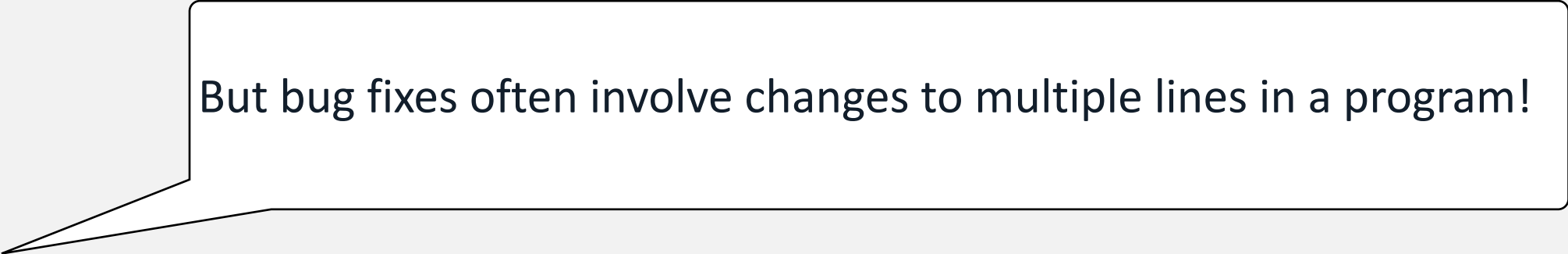
Correct, but these changes tend to be *spread around* the source code.

Complex faults consist of lots of “simple” localised faults.

They are not contiguous chunks of code.

If a test case can detect a lot of simple faults in a program, it will also be able to detect the more complex faults.

The Coupling Effect



But bug fixes often involve changes to multiple lines in a program!

Correct, but these changes tend to be *spread around* the source code.

Complex faults consist of lots of “simple” localised faults.

They are not contiguous chunks of code.

If a test case can detect a lot of simple faults in a program, it will also be able to detect the more complex faults.

Can we measure the adequacy of a test set in terms of its ability to detect “simple” faults?

A hand holding a pen is positioned over a document, with a blue overlay covering the entire image. The text 'Mutation Testing' is centered in white.

Mutation Testing

Mutations

Very minor changes to source code.

Simulated faults made by the competent programmer.

A mutated program is called a **mutant**.

Mutants are generated by **mutation operators**.

Operators might include:

Replacing a '+' with a '*'

Replacing a variable 'x' with a different variable 'y'.

Deleting a statement.

...

A large number of mutation operators have been developed over the years, for different kinds of programming languages.

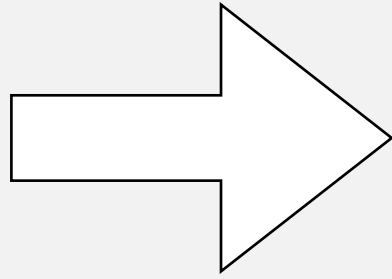


Example

```
x = x + 1;  
if (x > 0)  
  y = y + 1;  
else  
  y = y + 2
```

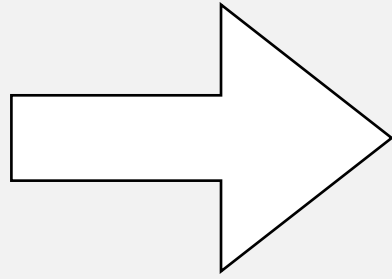
Example

```
x = x + 1;  
if (x > 0)  
  y = y + 1;  
else  
  y = y + 2
```



Example

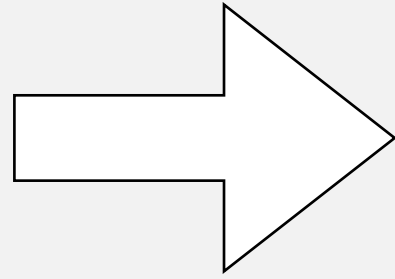
```
x = x + 1;  
if (x > 0)  
  y = y + 1;  
else  
  y = y + 2
```



```
x = x + 1;  
if (x > 0)  
  y = y - 1;  
else  
  y = y + 2
```


Example

```
x = x + 1;  
if (x > 0)  
  y = y + 1;  
else  
  y = y + 2
```

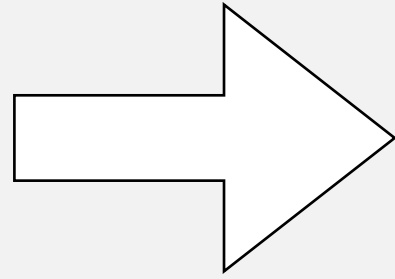


```
x = x + 1;  
if (x > 0)  
  y = y - 1;  
else  
  y = y + 2
```

```
x = x + 1;  
if (x >= 0)  
  y = y + 1;  
else  
  y = y + 2
```

Example

```
x = x + 1;  
if (x > 0)  
  y = y + 1;  
else  
  y = y + 2
```



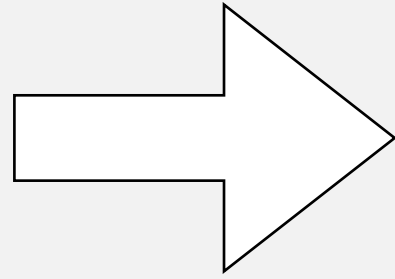
```
x = x + 1;  
if (x > 0)  
  y = y - 1;  
else  
  y = y + 2
```

```
x = x + 1;  
if (x >= 0)  
  y = y + 1;  
else  
  y = y + 2
```

```
x = x + 1;  
if (y > 0)  
  y = y + 1;  
else  
  y = y + 2
```

Example

```
x = x + 1;  
if (x > 0)  
  y = y + 1;  
else  
  y = y + 2
```



```
x = x + 1;  
if (x > 0)  
  y = y - 1;  
else  
  y = y + 2
```

```
x = x + 1;  
if (y > 0)  
  y = y + 1;  
else  
  y = y + 2
```

```
x = x + 1;  
if (x >= 0)  
  y = y + 1;  
else  
  y = y + 2
```

```
x = x / 1;  
if (x > 0)  
  y = y + 1;  
else  
  y = y + 2
```

Killing a Mutant

Killing a Mutant

A typical mutation testing setup:

A test set T

This is what we want to evaluate - how good is it at finding bugs?

A program under test p

A mutated version of the program (a mutant) p'

There are usually hundreds or even thousands of mutants.

Killing a Mutant

A typical mutation testing setup:

A test set T

This is what we want to evaluate - how good is it at finding bugs?

A program under test p

A mutated version of the program (a mutant) p'

There are usually hundreds or even thousands of mutants.

A mutant is **killed** if:

For at least one test case $t \in T$, the output from $p(t)$ is different from $p'(t)$.

Killing a Mutant

A typical mutation testing setup:

A test set T

This is what we want to evaluate - how good is it at finding bugs?

A program under test p

A mutated version of the program (a mutant) p'

There are usually hundreds or even thousands of mutants.

A mutant is **killed** if:

For at least one test case $t \in T$, the output from $p(t)$ is different from $p'(t)$.

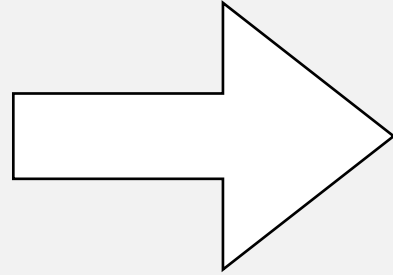
Example 1

```
x = x + 1;  
if (x > 0)  
  y = y + 1;  
else  
  y = y + 2
```

Test	x	y
T1	-1	0
T2	-3	-2
T3	11	6
T4	-5	2

Example 1

```
x = x + 1;  
if (x > 0)  
  y = y + 1;  
else  
  y = y + 2
```

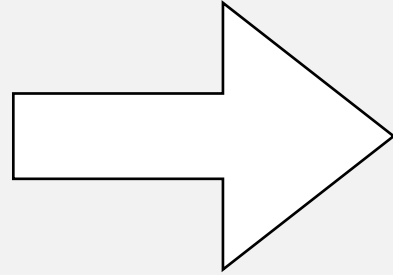


```
x = x + 1;  
if (x > 0)  
  y = y - 1;  
else  
  y = y + 2
```

Test	x	y
T1	-1	0
T2	-3	-2
T3	11	6
T4	-5	2

Example 1

```
x = x + 1;  
if (x > 0)  
  y = y + 1;  
else  
  y = y + 2
```



```
x = x + 1;  
if (x > 0)  
  y = y - 1;  
else  
  y = y + 2
```

Test	x	y
T1	-1	0
T2	-3	-2
T3	11	6
T4	-5	2

Test	Orig. output x	Orig. output y	Mutant output x	Mutant output y
T1	0	2	0	2
T2	-2	0	-2	0
T3	12	7	12	5
T4	-4	4	-4	4

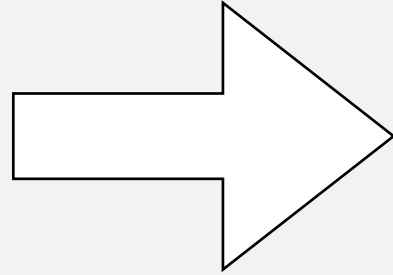
Example 2

```
x = x + 1;  
if (x > 0)  
  y = y + 1;  
else  
  y = y + 2
```

Test	x	y
T1	-1	-1
T2	-3	-3
T3	11	11
T4	-5	-5

Example 2

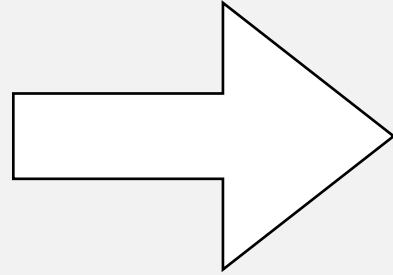
```
x = x + 1;  
if (x > 0)  
  y = y + 1;  
else  
  y = y + 2
```



Test	x	y
T1	-1	-1
T2	-3	-3
T3	11	11
T4	-5	-5

Example 2

```
x = x + 1;  
if (x > 0)  
  y = y + 1;  
else  
  y = y + 2
```

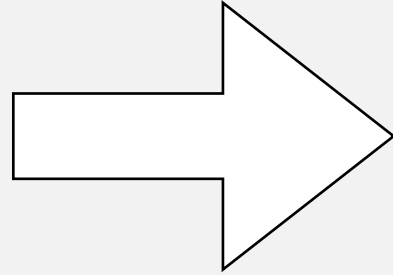


```
x = x + 1;  
if (x >= 0)  
  y = y + 1;  
else  
  y = y + 2
```

Test	x	y
T1	-1	-1
T2	-3	-3
T3	11	11
T4	-5	-5

Example 2

```
x = x + 1;  
if (x > 0)  
  y = y + 1;  
else  
  y = y + 2
```



```
x = x + 1;  
if (x >= 0)  
  y = y + 1;  
else  
  y = y + 2
```

Test	x	y
T1	-1	-1
T2	-3	-3
T3	11	11
T4	-5	-5

Test	Orig. output x	Orig. output y	Mutant output x	Mutant output y
T1	0	1	0	0
T2	-2	-1	-2	-1
T3	12	12	12	12
T4	-4	-3	-4	-3

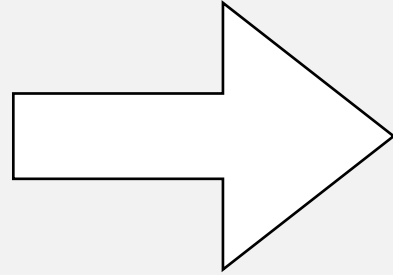
Example 3

```
x = x + 1;  
if (x > 0)  
    y = y + 1;  
else  
    y = y + 2
```

Test	x	y
T1	-1	-1
T2	-3	-3
T3	11	11
T4	-5	-5

Example 3

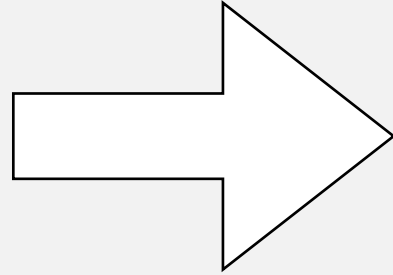
```
x = x + 1;  
if (x > 0)  
  y = y + 1;  
else  
  y = y + 2
```



Test	x	y
T1	-1	-1
T2	-3	-3
T3	11	11
T4	-5	-5

Example 3

```
x = x + 1;  
if (x > 0)  
  y = y + 1;  
else  
  y = y + 2
```

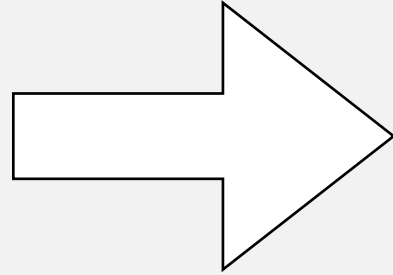


```
x = x + 1;  
if (y > 0)  
  y = y + 1;  
else  
  y = y + 2
```

Test	x	y
T1	-1	-1
T2	-3	-3
T3	11	11
T4	-5	-5

Example 3

```
x = x + 1;  
if (x > 0)  
  y = y + 1;  
else  
  y = y + 2
```



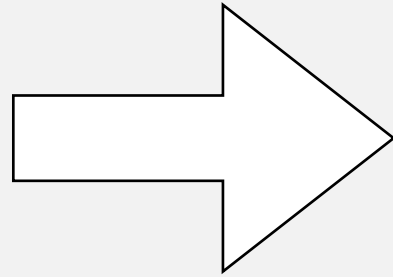
```
x = x + 1;  
if (y > 0)  
  y = y + 1;  
else  
  y = y + 2
```

Test	x	y
T1	-1	-1
T2	-3	-3
T3	11	11
T4	-5	-5

Test	Orig. output x	Orig. output y	Mutant output x	Mutant output y
T1	0	1	0	1
T2	-2	-1	-2	-1
T3	12	12	12	12
T4	-4	-3	-4	-3

Example 3

```
x = x + 1;  
if (x > 0)  
  y = y + 1;  
else  
  y = y + 2
```



```
x = x + 1;  
if (y > 0)  
  y = y + 1;  
else  
  y = y + 2
```

Not detected by the test set.

Could be detected with better tests.

Need tests where x and y are different!

Test	x	y
T1	-1	-1
T2	-3	-3
T3	11	11
T4	-5	-5

Test	Orig. output x	Orig. output y	Mutant output x	Mutant output y
T1	0	1	0	1
T2	-2	-1	-2	-1
T3	12	12	12	12
T4	-4	-3	-4	-3

Using Mutants to Assess Test Set Performance

Kill Matrix

Which mutants were killed by which tests?

	M1	M2	M3	M4
T1	X			X
T2	X		X	X
T3	X			
T4			X	

Mutation score

Proportion of mutants killed by whole test set.

$$\frac{3}{4} = 0.75$$

Using Mutants to Assess Test Set Performance

Kill Matrix

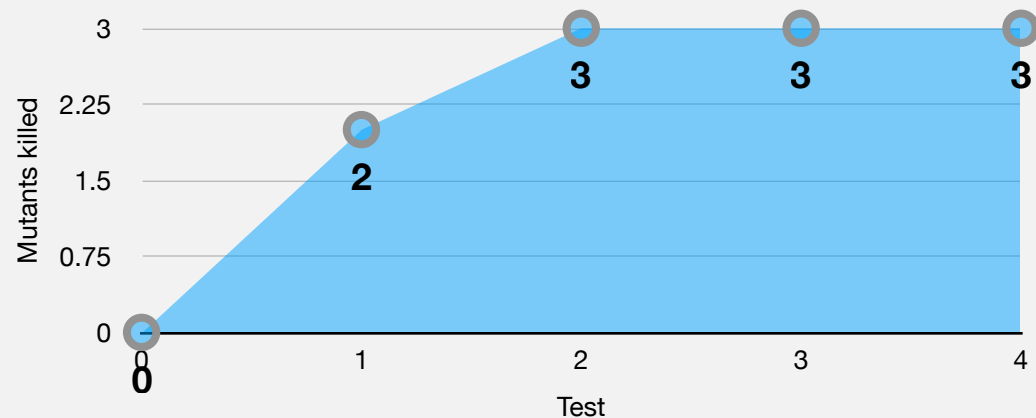
Which mutants were killed by which tests?

	M1	M2	M3	M4
T1	X			X
T2	X		X	X
T3	X			
T4			X	

Mutation score

Proportion of mutants killed by whole test set.

$$\frac{3}{4} = 0.75$$



Using Mutants to Assess Test Set Performance

Kill Matrix

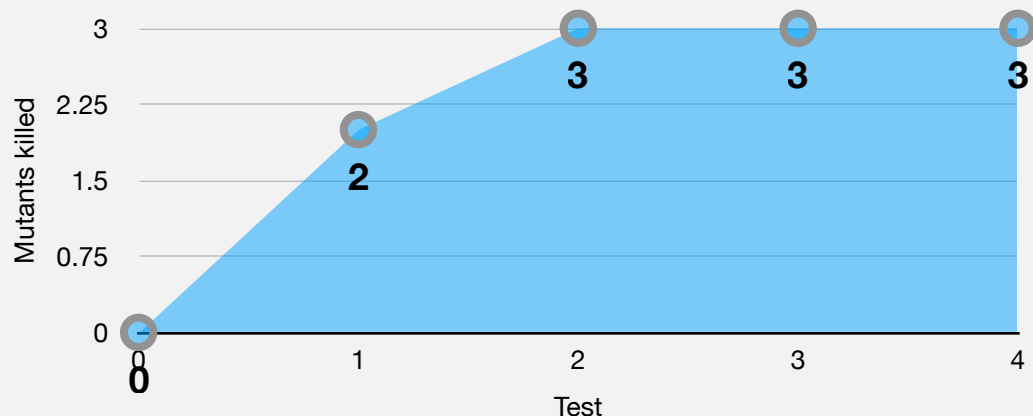
Which mutants were killed by which tests?

	M1	M2	M3	M4
T1	X			X
T2	X		X	X
T3	X			
T4			X	

Mutation score

Proportion of mutants killed by whole test set.

$$\frac{3}{4} = 0.75$$



Can use same APFD measure as in Regression Testing.

$$APFD = 1 - \frac{TF_1 + TF_2 + \dots + TF_f}{nf} + \frac{1}{2n}$$

Where

TF_i is the position of the first test that kills mutant i .

n is the number of tests.

f is the number of faults.

If a fault is never detected, the value is $n + 1$

For example on the left:

$$\begin{aligned} APFD &= 1 - \frac{1 + 5 + 2 + 1}{4 \cdot 4} + \frac{1}{2 \cdot 4} \\ &= 0.5625 \end{aligned}$$



Challenges in Mutation Testing

Equivalent Mutants

A mutant p' of p is equivalent if no possible input can kill p' .

p' is *syntactically* different, but semantically equivalent.



Equivalent Mutants

A mutant p' of p is equivalent if no possible input can kill p' .

p' is *syntactically* different, but semantically equivalent.

```
...  
x = 2;  
x = x + x;  
...
```

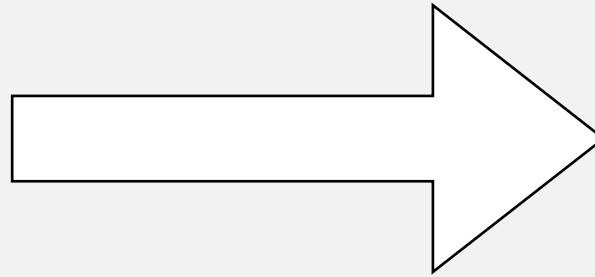


Equivalent Mutants

A mutant p' of p is equivalent if no possible input can kill p' .

p' is *syntactically* different, but semantically equivalent.

```
...  
x = 2;  
x = x + x;  
...
```



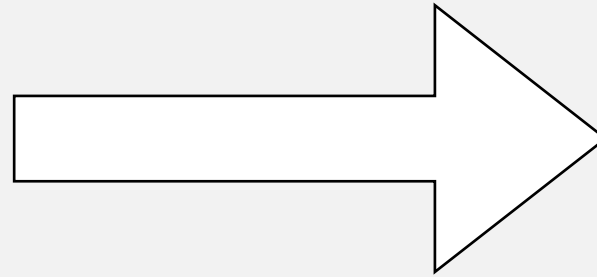
Equivalent Mutants

A mutant p' of p is equivalent if no possible input can kill p' .

p' is *syntactically* different, but semantically equivalent.



```
...  
x = 2;  
x = x + x;  
...
```



```
...  
x = 2;  
x = x * x;  
...
```

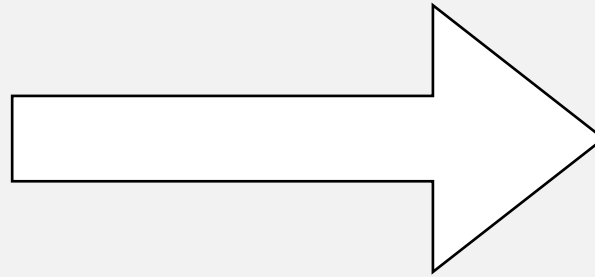
Equivalent Mutants

A mutant p' of p is equivalent if no possible input can kill p' .

p' is *syntactically* different, but semantically equivalent.



```
...  
x = 2;  
x = x + x;  
...
```



```
...  
x = 2;  
x = x * x;  
...
```

```
...  
if x + 1 mod 2 == 0;  
    y = x  
...
```

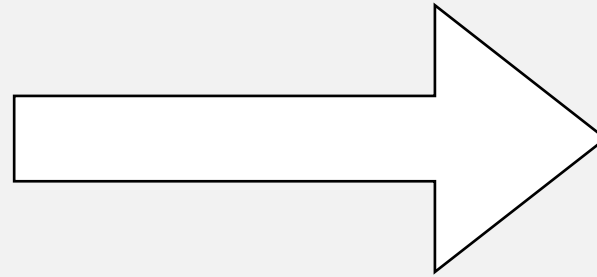
Equivalent Mutants

A mutant p' of p is equivalent if no possible input can kill p' .

p' is *syntactically* different, but semantically equivalent.

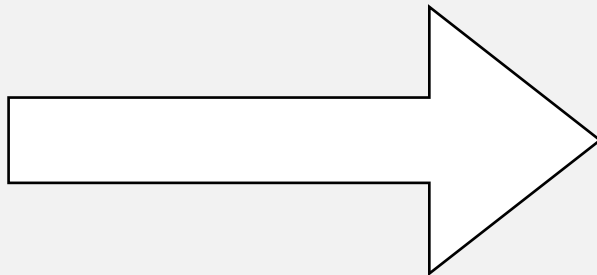


```
...  
x = 2;  
x = x + x;  
...
```



```
...  
x = 2;  
x = x * x;  
...
```

```
...  
if x + 1 mod 2 == 0;  
    y = x  
...
```



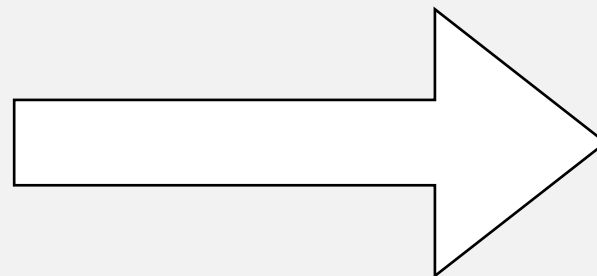
Equivalent Mutants

A mutant p' of p is equivalent if no possible input can kill p' .

p' is *syntactically* different, but semantically equivalent.

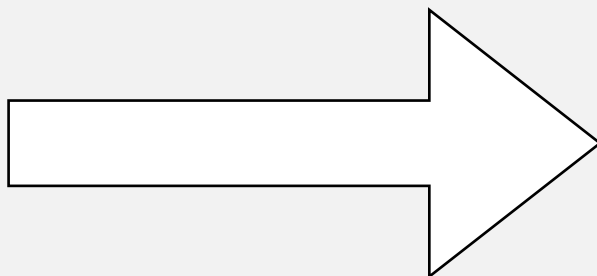


```
...  
x = 2;  
x = x + x;  
...
```



```
...  
x = 2;  
x = x * x;  
...
```

```
...  
if x + 1 mod 2 == 0;  
    y = x  
...
```



```
...  
if x - 1 mod 2 == 0;  
    y = x  
...
```

Why are equivalent mutants problematic?

Why are equivalent mutants problematic?

Generally impossible to determine that a mutant is equivalent.

Equivalence is *undecidable*.

Why are equivalent mutants problematic?

Generally impossible to determine that a mutant is equivalent.

Equivalence is *undecidable*.

Can undermine trustworthiness of a mutation score.

Is it low because the test set is genuinely poor?

... Or is it low because there are lots of equivalent mutants?

Why are equivalent mutants problematic?

Generally impossible to determine that a mutant is equivalent.

Equivalence is *undecidable*.

Can undermine trustworthiness of a mutation score.

Is it low because the test set is genuinely poor?

... Or is it low because there are lots of equivalent mutants?

Typically necessary to fall back on a human tester to decide on this.

Performance



Performance

Each mutant needs to:

Be **compiled** into a runnable program.

Tested against the full test set.



Performance

Each mutant needs to:

Be **compiled** into a runnable program.

Tested against the full test set.

Several factors that can cost time.

Number of mutation operators to apply.

The Proteum tool for C has 103 possible mutation operators.

MuJava adds a further 24 class-level operators.

Amount of code they can be applied to.

Size of the test set.

Run-time for each test-case.



Mutation testing tools

PIT for Java (<https://pitest.org/>)

Mutmut for Python (<https://github.com/boxed/mutmut>)

Mutest-rs for Rust (<https://github.com/zalanlevai/mutest-rs>)

Stryker Mutator for .NET (<https://stryker-mutator.io/>)

Mull for C / C++ (<https://mull-project.com/>)



Code Defenders

A Mutation Testing Game

José Miguel Rojas



Gameplay

Class Under Test

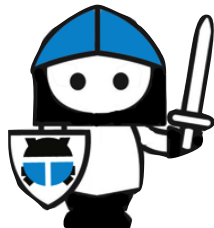
```
public class Arithmetics {  
    public int abs(int x) {  
        if (x >= 0)  
            return x;  
        else  
            return -x;  
    }  
}
```



Gameplay

Class Under Test

```
public class Arithmetics {  
    public int abs(int x) {  
        if (x >= 0)  
            return x;  
        else  
            return -x;  
    }  
}
```



Attackers



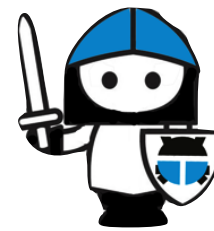
Gameplay

Class Under Test

```
public class Arithmetics {  
    public int abs(int x) {  
        if (x >= 0)  
            return x;  
        else  
            return -x;  
    }  
}
```



Attackers



Defenders



Gameplay

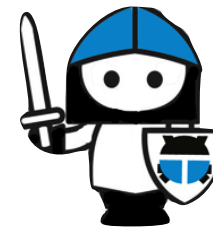
Class Under Test

```
public class Arithmetics {  
    public int abs(int x) {  
        if (x < 0)  
            return x;  
        else  
            return -x;  
    }  
}
```



Attackers

```
public class Arithmetics {  
    public int abs(int x) {  
        if (x >= 0)  
            return x;  
        else  
            return -x;  
    }  
}
```



Defenders



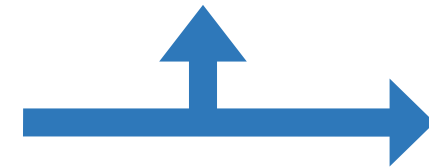
Gameplay

Class Under Test

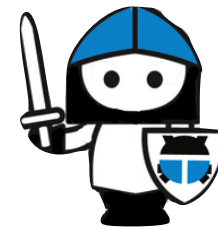
```
public class Arithmetics {  
    public int abs(int x) {  
        if (x < 0)  
            return x;  
        else  
            return -x;  
    }  
}
```



Attackers



```
public class Arithmetics {  
    public int abs(int x) {  
        if (x >= 0)  
            return x;  
        else  
            return -x;  
    }  
}
```



Defenders



Gameplay

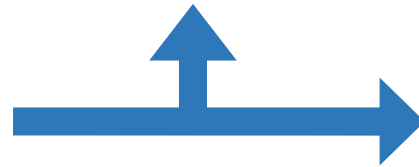
Class Under Test

```
public class Arithmetics {  
    public int abs(int x) {  
        if (x < 0)  
            return x;  
        else  
            return -x;  
    }  
}
```



Attackers

```
public class Arithmetics {  
    public int abs(int x) {  
        if (x >= 0)  
            return x;  
        else  
            return -x;  
    }  
}
```



```
public class TestArithmetics {  
    @Test  
    public void testAbs() {  
        Arithmetics a;  
        a = new Arithmetics();  
        assertEquals(1, a.abs(-1));  
    }  
}
```



Defenders



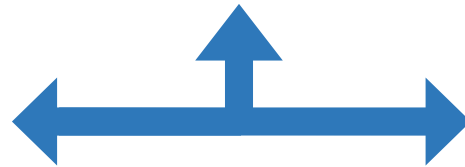
Gameplay

Class Under Test

```
public class Arithmetics {  
    public int abs(int x) {  
        if (x >= 0)  
            return x;  
        else  
            return -x;  
    }  
}
```

```
public class Arithmetics {  
    public int abs(int x) {  
        if (x < 0)  
            return x;  
        else  
            return -x;  
    }  
}
```

```
public class TestArithmetics {  
    @Test  
    public void testAbs() {  
        Arithmetics a;  
        a = new Arithmetics();  
        assertEquals(1, a.abs(-1));  
    }  
}
```



Attackers



Defenders



Gameplay

Class Under Test

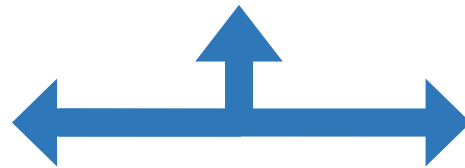
Score points for
surviving mutants

```
public class Arithmetics {  
    public int abs(int x) {  
        if (x < 0)  
            return x;  
        else  
            return -x;  
    }  
}
```



Attackers

```
public class Arithmetics {  
    public int abs(int x) {  
        if (x >= 0)  
            return x;  
        else  
            return -x;  
    }  
}
```



```
public class TestArithmetics {  
    @Test  
    public void testAbs() {  
        Arithmetics a;  
        a = new Arithmetics();  
        assertEquals(1, a.abs(-1));  
    }  
}
```



Defenders



Gameplay

Class Under Test

Score points for
surviving mutants

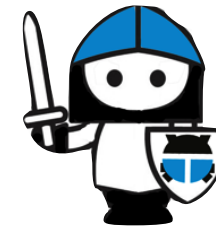
```
public class Arithmetics {  
    public int abs(int x) {  
        if (x < 0)  
            return x;  
        else  
            return -x;  
    }  
}
```



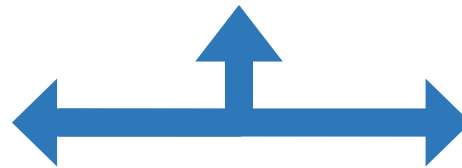
Attackers

Score points for
effective tests

```
public class TestArithmetics {  
    @Test  
    public void testAbs() {  
        Arithmetics a;  
        a = new Arithmetics();  
        assertEquals(1, a.abs(-1));  
    }  
}
```



Defenders





Gameplay

Class Under Test

Score points for
surviving mutants

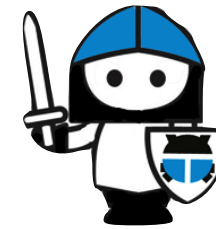
```
public class Arithmetics {  
    public int abs(int x) {  
        if (x < 0)  
            return x;  
        else  
            return -x;  
    }  
}
```



Attackers

Score points for
effective tests

```
public class TestArithmetics {  
    @Test  
    public void testAbs() {  
        Arithmetics a;  
        a = new Arithmetics();  
        assertEquals(1, a.abs(-1));  
    }  
}
```



Defenders

Equivalent Mutant Duels



Gameplay

Class Under Test

Score points for
surviving mutants

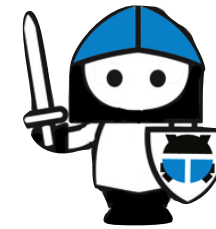
```
public class Arithmetics {  
    public int abs(int x) {  
        if (x < 0)  
            return x;  
        else  
            return -x;  
    }  
}
```



Attackers

Score points for
effective tests

```
public class TestArithmetics {  
    @Test  
    public void testAbs() {  
        Arithmetics a;  
        a = new Arithmetics();  
        assertEquals(1, a.abs(-1));  
    }  
}
```



Defenders

```
public class Arithmetics {  
    public int abs(int x) {  
        if (x >= 0)  
            return x;  
        else  
            return -x;  
    }  
}
```

equivalent!

Equivalent Mutant Duels



Gameplay

Class Under Test

Score points for
surviving mutants

```
public class Arithmetics {  
    public int abs(int x) {  
        if (x < 0)  
            return x;  
        else  
            return -x;  
    }  
}
```



Attackers

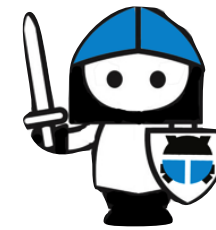
no way! here is a
killing test!

Class Under Test

```
public class Arithmetics {  
    public int abs(int x) {  
        if (x >= 0)  
            return x;  
        else  
            return -x;  
    }  
}
```

Score points for
effective tests

```
public class TestArithmetics {  
    @Test  
    public void testAbs() {  
        Arithmetics a;  
        a = new Arithmetics();  
        assertEquals(1, a.abs(-1));  
    }  
}
```



Defenders

equivalent!

Equivalent Mutant Duels



Gameplay

Class Under Test

Score points for
surviving mutants

```
public class Arithmetics {  
  public int abs(int x) {  
    if (x < 0)  
      return x;  
    else  
      return -x;  
  }  
}
```

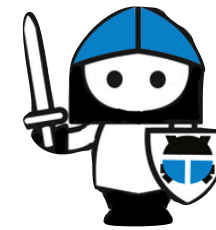


Attackers

no way! here is a
killing test!

Score points for
effective tests

```
public class TestArithmetics {  
  @Test  
  public void testAbs() {  
    Arithmetics a;  
    a = new Arithmetics();  
    assertEquals(1, a.abs(-1));  
  }  
}
```



Defenders

equivalent!

Equivalent Mutant Duels



Gameplay

Class Under Test

Score points for
surviving mutants

```
public class Arithmetics {  
    public int abs(int x) {  
        if (x < 0)  
            return x;  
        else  
            return -x;  
    }  
}
```



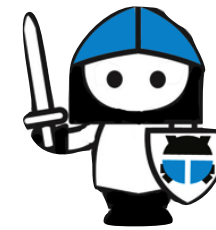
Attackers

no way! here is a
killing test!

oh no! :(

Score points for
effective tests

```
public class TestArithmetics {  
    @Test  
    public void testAbs() {  
        Arithmetics a;  
        a = new Arithmetics();  
        assertEquals(1, a.abs(-1));  
    }  
}
```



Defenders

equivalent!

Equivalent Mutant Duels



Play Modes

Game #3686 **Melee** **Player** 43min Scoreboard Timeline Gradle Export Feedback Editor Mode: default Chat

Create a mutant here Enemy Coverage Reset Attack Write a new JUnit test here Clone previous test Defend

```
33 public boolean isFull() {
34     return numRiders == capacity;
35 }
36
37 public void addRiders(int numEntering) {
38     if (numRiders + numEntering <= capacity) {
39         numRiders = numRiders + numEntering;
40     } else {
41         numRiders = capacity;
42     }
43 }
44
45 public void goUp() {
46     if (currentFloor < topFloor)
47         currentFloor++;
48 }
49
50 public void goDown() {
51     if (currentFloor > 0)
52         currentFloor--;
53 }
54
55 public void call(int floor) {
56     if (floor >= 0 && floor <= topFloor) {
57         while (floor != currentFloor) {
58             if (floor > currentFloor)
59                 goUp();
60             else
61                 goDown();
62         }
63     }
64 }
65 }
```

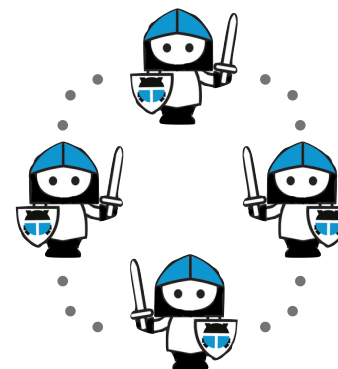
Live Killed Claimed Equivalent Equivalent
Mutant/Test restrictions: **Moderate**

Existing Mutants All Alive Killed Claimed Equivalent Equivalent

All Mutants	
Mutants outside methods	
Lift(int)	
Lift(int, int)	
getTopFloor()	
getCurrentFloor()	
getCapacity()	

JUnit Tests

All Tests	
Lift(int)	
Lift(int, int)	
getTopFloor()	
getCurrentFloor()	
getCapacity()	
getNumRiders()	



Multi-player



Code Coverage and Mutants

```
37     public void addRiders(int numEntering) {
38         if (numRiders + numEntering <= capacity) {
39             numRiders = numRiders + numEntering;
40         } else {
41             numRiders = capacity;
42         }
43     }
44
45     public void goUp() {
46         if (currentFloor < topFloor)
47             currentFloor++;
48     }
49
50     public void goDown() {
51         if (currentFloor > 0)
52             currentFloor--;
53     }
54
55     public void call(int floor) {
56         if (floor >= 0 && floor <= topFloor) {
57             while (floor != currentFloor) {
58                 if (floor > currentFloor)
59                     goUp();
60                 else
61                     goDown();
```

ayer

er

er



Code Coverage and Mutants

```
37     public void addRiders(int numEntering) {  
38         if (numRiders + numEntering <= capacity) {  
39             numRiders = numRiders + numEntering;  
40         } else {  
41             numRiders = capacity;  
42         }  
43     }
```

```
45     public void goUp() {  
46         if (currentFloor < topFloor)  
47             currentFloor++;  
48     }
```

```
50     public void goDown() {  
51         if (currentFloor > 0)  
52             currentFloor--;  
53     }
```

```
55     public void call(int floor) {  
56         if (floor >= 0 && floor <= topFloor) {  
57             while (floor != currentFloor) {  
58                 if (floor > currentFloor)  
59                     goUp();  
60                 else  
61                     goDown();
```

The greener the line,
the more tests that cover it!

ayer

er



Code Coverage and Mutants

```
37 public void addRiders(int numEntering) {  
38     if (numRiders + numEntering <= capacity) {  
39         numRiders = numRiders + numEntering;  
40     }  
41 }  
42  
43  
44  
45 public void goUp() {  
46     if (currentFloor < topFloor)  
47         currentFloor++;  
48 }  
49  
50 public void goDown() {  
51     if (currentFloor > 0)  
52         currentFloor--;  
53 }  
54  
55 public void call(int floor) {  
56     if (floor >= 0 && floor <= topFloor) {  
57         while (floor != currentFloor) {  
58             if (floor > currentFloor)  
59                 goUp();  
60             else  
61                 goDown();
```

Mutant markers:

 Live  Killed  Claimed Equivalent  Equivalent

The greener the line,
the more tests that cover it!

ayer

er



Scoreboard

Scoreboard				
7  3				
Attackers	Mutants	Alive / Killed / Equivalent	Duels Won / Lost / Ongoing	Total Points
user2	4	3 / 1 / 0	0 / 0 / 1	7
Attacking Team	4	3 / 1 / 0	0 / 0 / 1	7
Defenders	Tests	Mutants Killed	Duels Won / Lost / Ongoing	Total Points
user1	2	1	0 / 0 / 1	3
Defending Team	2	1	0 / 0 / 1	3
Close				



Today's Session – Sign up

Go to <https://code-defenders.org/>

Click on **Log in or Sign up** to create an account and sign in

You should then see this page:

The screenshot shows a web browser window with the URL `code-defenders.org/games/overview`. The page has a blue header with the Code Defenders logo, navigation tabs for 'Multiplayer' and 'Puzzles', and a user profile 'shefStud'. The main content area is titled 'My Games' and contains a table with columns: ID, Creator, Class, Players, and Level. A message states 'You are currently not active in any games.' Below this are two buttons: 'Create battleground game' and 'Create melee game'. Further down is a section titled 'Open Battleground Games' with a table showing game details.

ID	Creator	Class	Attackers	Defenders	Level
> 135	sandya123	Lift	44 Join	41 Join	Hard



Today's Session – Puzzles

Next, go to **Puzzles** and complete **as many puzzles as you can in 15 minutes** (puzzle tiles turn green when completed):

Puzzles

Class Under Test

```
1 public class Puzzle1 {
2     public int multiply(int x, int y) {
3         return x * y;
4     }
5 }
```

Write a new JUnit test here

```
1 import org.junit.Test;
2
3 import static org.junit.Assert.*;
4
5 import static org.hamcrest.Matchers.*;
6
7 public class Puzzle1Test {
8     @Test(timeout = 4000)
9     public void test() throws Throwable {
10         // Test here
11     }
12 }
```

Existing Mutants

All Mutants

JUnit Tests

All Tests

NEXT PUZZLE:
Beginner, Puzzle 3
Write a mutant which evades all the tests

Beginner

Puzzle 1
Write a mutant which evades all the tests

Puzzle 2
Write a test which kills the remaining mutant

Puzzle 3
Write a mutant which evades all the tests

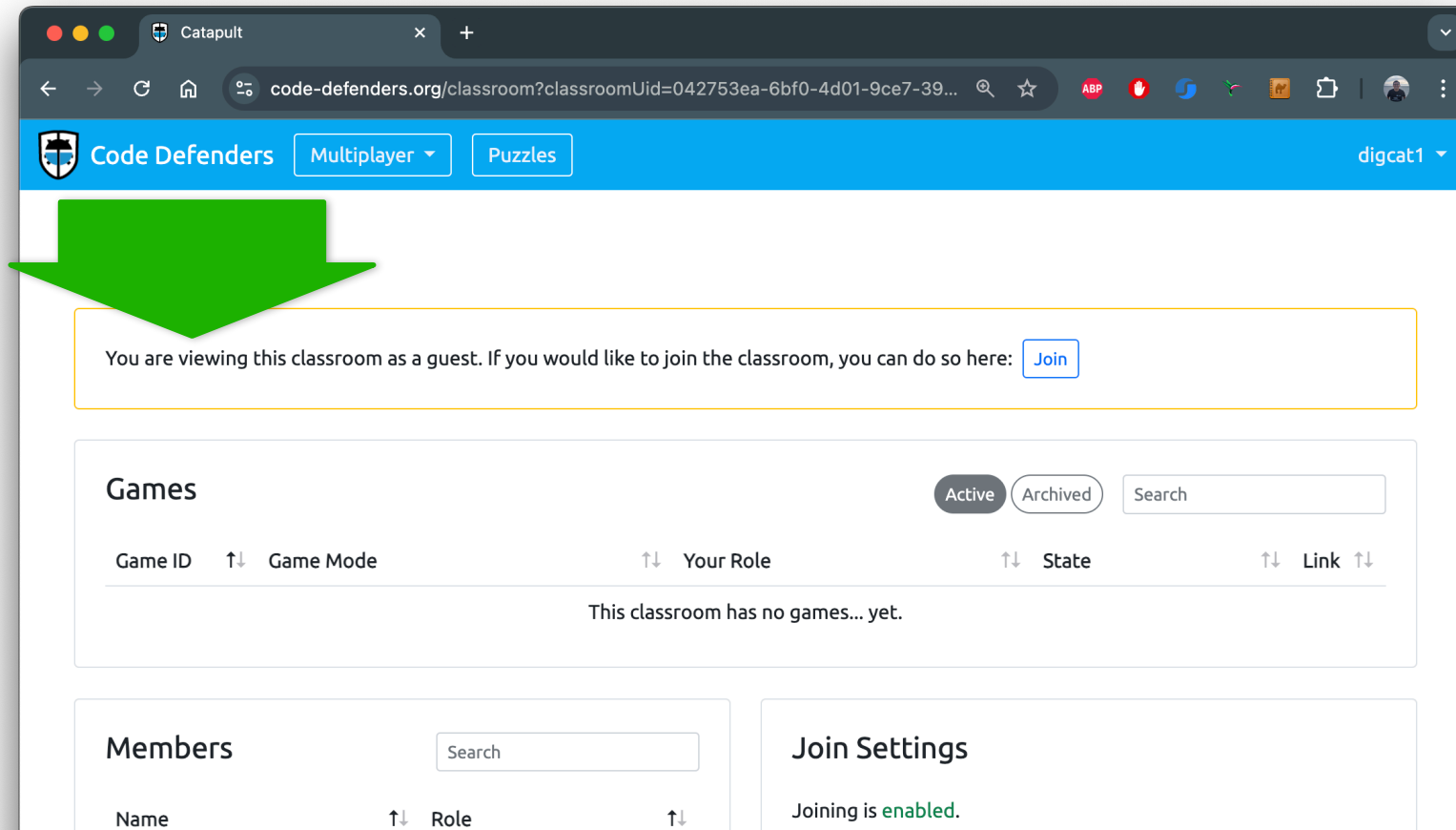
Puzzle 4
Write a test which kills the remaining mutant

Puzzle 5
Write a mutant which evades all the tests



Today's Session – Classroom

Next, join the [COM3529 2025 Code Defenders classroom](https://code-defenders.org/classroom?classroomUid=042753ea-6bf0-4d01-9ce7-39...)



The screenshot shows a web browser window with the URL `code-defenders.org/classroom?classroomUid=042753ea-6bf0-4d01-9ce7-39...`. The page has a blue header with the "Code Defenders" logo, "Multiplayer" and "Puzzles" buttons, and a user profile "digcat1". A yellow box highlights a message: "You are viewing this classroom as a guest. If you would like to join the classroom, you can do so here: [Join](#)". Below this is a "Games" section with tabs for "Active" and "Archived", a search bar, and a table with columns: Game ID, Game Mode, Your Role, State, and Link. The table is empty with the text "This classroom has no games... yet." Below the games section are "Members" and "Join Settings" sections. The "Members" section has a search bar and a table with columns: Name, Role. The "Join Settings" section shows "Joining is enabled."

Code Defenders Multiplayer Puzzles digcat1

You are viewing this classroom as a guest. If you would like to join the classroom, you can do so here: [Join](#)

Games Active Archived Search

Game ID	Game Mode	Your Role	State	Link
This classroom has no games... yet.				

Members Search

Name	Role
------	------

Join Settings

Joining is **enabled**.



Today's Session – Classroom

Once you have joined the classroom, wait to be added to a game and click on the game link when its **State** changes to **Running: XXmin left**.

The screenshot shows a web browser window with the URL `code-defenders.org/classroom?classroomUid=66b2ac48-1343-4708-8df9-bbbc6128296f`. The page title is "TUOS COM3529". The navigation bar includes "Code Defenders", "Multiplayer", and "Puzzles". The user "shefStud" is logged in.

The "Games" section is highlighted with a green border. It contains a table with the following data:

Game ID	Game Mode	Your Role	State	Link
3686	Melee	Player	Not Started	Link

Below the "Games" section, there are two other sections: "Members" and "Join Settings".

Members

Name	Role
------	------

Join Settings

Joining is **enabled**.
Visibility is **private**.



Today's Session – Game on!

Game 3686 (Player) x +

code-defenders.org/meleegame?gameId=3686

Code Defenders Multiplayer Puzzles shefStud

Game #3686 Melee Player 1h Scoreboard Timeline Gradle Export Feedback Editor Mode: default Chat 0

Create a mutant here Enemy Coverage Reset Attack

```
1 public class Lift {
2
3     private int topFloor;
4     private int currentFloor = 0; // default
5     private int capacity = 10; // default
6     private int numRiders = 0; // default
7
8     public Lift(int highestFloor) {
9         topFloor = highestFloor;
10    }
11
12    public Lift(int highestFloor, int maxRiders) {
13        this(highestFloor);
14        capacity = maxRiders;
15    }
16
17    public int getTopFloor() {
18        return topFloor;
19    }
20
21    public int getCurrentFloor() {
22        return currentFloor;
23    }
24
25    public int getCapacity() {
26        return capacity;
27    }
28
29    public int getNumRiders() {
30        return numRiders;
31    }
32
33    public boolean isFull() {
34        return numRiders == capacity;
35    }
36 }
```

Live Killed Claimed Equivalent Equivalent
Mutant/Test restrictions: Moderate

Write a new JUnit test here Clone previous test Defend

```
1 import org.junit.Test;
2
3 import static org.junit.Assert.*;
4 import static org.hamcrest.MatcherAssert.assertThat;
5 import static org.hamcrest.Matchers.*;
6
7 public class TestLift {
8     @Test(timeout = 4000)
9     public void test() throws Throwable {
10         // test here!
11     }
12 }
```

Existing Mutants All Alive Killed Claimed Equivalent Equivalent

All Mutants	▼
Mutants outside methods	▼
Lift(int)	▼
Lift(int, int)	▼

JUnit Tests

All Tests	▼
Lift(int)	▼
Lift(int, int)	▼
getTopFloor()	▼

You will play both as **attacker** (creating mutants) and as **defender** (creating tests) at the same time.
Have fun!



Lift Class

```
public class Lift {

    private int topFloor;
    private int currentFloor = 0; // default
    private int capacity = 10;    // default
    private int numRiders = 0;    // default

    public Lift(int highestFloor) {
        topFloor = highestFloor;
    }

    public Lift(int highestFloor, int maxRiders) {
        this(highestFloor);
        capacity = maxRiders;
    }

    public int getTopFloor() {
        return topFloor;
    }

    public int getCurrentFloor() {
        return currentFloor;
    }

    public int getCapacity() {
        return capacity;
    }

    public int getNumRiders() {
        return numRiders;
    }
}
```

```
    public boolean isFull() {
        return numRiders == capacity;
    }

    public void addRiders(int numEntering) {
        if (numRiders + numEntering <= capacity) {
            numRiders = numRiders + numEntering;
        } else {
            numRiders = capacity;
        }
    }

    public void goUp() {
        if (currentFloor < topFloor)
            currentFloor++;
    }

    public void goDown() {
        if (currentFloor > 0)
            currentFloor--;
    }

    public void call(int floor) {
        if (floor >= 0 && floor <= topFloor) {
            while (floor != currentFloor) {
                if (floor > currentFloor)
                    goUp();
                else
                    goDown();
            }
        }
    }
}
```



Example Test for Lift Class

```
@Test
public void testEmptyLift() {
    Lift l = new Lift(5, 10);
    l.call(3);
    assertFalse(l.isFull());
    assertEquals(3, l.getCurrentFloor());
}
```



Stack Class

```
public class Stack<T> {  
    private int capacity = 10;  
    private int pointer = 0;  
    private T[] objects = (T[]) new Object[capacity];  
  
    public void push(T o) {  
        if(pointer >= capacity)  
            throw new IllegalStateException("Stack exceeded capacity!");  
        objects[pointer++] = o;  
    }  
  
    public T pop() {  
        if(pointer <= 0)  
            throw new IllegalStateException("Stack empty");  
        return objects[--pointer];  
    }  
  
    public boolean isEmpty() {  
        return pointer <= 0;  
    }  
}
```




Example Tests for Stack

```
@Test
public void testStackException() {
    Stack<Integer> s = new Stack<Integer>();
    assertTrue(s.isEmpty());
    try {
        s.pop();
        fail("pop on an empty list should fail");
    } catch (IllegalStateException e) {
        // It's all good, exception was expected
    }
}
```

```
@Test
public void testEmptyStack() {
    Stack s = new Stack<String>();
    s.push("Hello");
    assertFalse(s.isEmpty());
}
```